

# מסמך סיכום עיצוב: מערכת אוטומטית לשיבוץ לוחות מבחנים

18 במאי 2026

## חלק I

### הרעיון המרכזי

המערכת נועדה לפתור בעיה אקדמית מורכבת של שיבוץ לוחות מבחנים תחת אילוצים. המערכת מקבלת כקלט קבצי טקסט המכילים את נתוני הקורסים באוניברסיטה, מסלולי הלימוד המבוקשים, ואת לוח השנה האקדמי הכולל ימי חג ומועדים אסורים. הפלט הסופי הוא הפקה של כל לוחות הזמנים האפשריים שהם תקפים לחלוטין ונקיים מהתנגשויות. מטרת מרכזיות:

1. חלוקת הבעיה לתתי בעיות:  
מיפוי וחלוקה דינמית של משימות השיבוץ לפי תקופות מבחנים מבודדות (למשל: סמסטר א' מועד א', או סמסטר ב' מועד ב') כדי למנוע עומס חישובי.
2. אילוצים: הבטחה ששני קורסי חובה השייכים לאותו מסלול לימודים ולאותה שנת לימוד לא ישובצו באותו היום.
3. אופטימיזציה של מרחב החיפוש: התמודדות יעילה עם הגידול המעריכי של התאמת קורסים לימים מבלי לגרום לקריסת זיכרון של המערכת.

## חלק II

### מבנה

קוד הפרויקט בנוי על בסיס תכנות מונחן עצמים בPython ומבוססת על תבנית Controller-Service-Repository, המפרידה בין שכבות הנתונים, הלוגיקה והפלט. מצורפים 2 קבצי UML: הראשון מראה את מימוש האלגוריתם והשני את תיאור הקלאסים והיחסים השונים ביניהם

1. שכבת ניהול וריצה מרכזית:  
Main: קובץ ההרצה הראשי של המערכת. הוא אחראי לקבוע את נתיבי קבצי הקלט (הקורסים, התאריכים ותוכניות הלימוד), לאתחל את מנהל האפליקציה הראשי ולהפעיל את זרימת המערכת. במקרה של תקלה כלשהי בקריאה או בריצה, שכבה זו תתפוס את השגיאה ותציג הודעה מרוכזת למשתמש.

AppController: הרכיב המרכזי שמנהל ומקשר בין כל חלקי המערכת בהתאם לדיאגרמת המחלקות. הוא מחזיק בתוכו את המופעים של קוראי הקבצים השונים ושל כותב הדוחות הכולל:  
אימות מוקדם של נתיבי הקבצים כדי לוודא שהם קיימים ואינם ריקים.

תיאום תהליך קריאת הנתונים וסינון הקורסים הרלוונטיים.  
בניית אינדקס האילוצים וקטלוג התקופות.  
אתחול מנוע השיבוץ, הפעלת תהליך יצירת לוחות הזמנים, והעברת התוצאות הסופיות לרכיב כתיבת הדוחות לצורך שמירה בקובץ פלט.

## 2. שכבת עיבוד וקריאת הנתונים:

שכבה זו אחראית על קריאת קבצי הטקסט המתקבלים על ידי המשתמש: Dates, Programs, Courses

IFileParser: ממשק המגדיר את קריאת הקבצים  
CourseFileParser: קורא את נתוני הקורסים ומסנן בשלב מוקדם קורסים שאינם דורשים מבחן, כגון עבודות הגשה או נוכחות בלבד.  
ExamPeriodFileParser: מפענח את תאריכי ההתחלה והסיום של תקופת המבחנים ומזהה תאריכים אסורים (ימי שבת, חגים) כדי לבנות רשימה נקייה של ימים זמינים לשיבוץ.  
ProgramSelectionParser: טוען את מסלולי הלימוד שנבחרו ומגביל את הבחירה לעד 5 תוכניות לימוד על פי דרישות המערכת.

## 3. שכבת מודלים:

Course, ProgramRequirement: מייצגים את הקלאסים של הקורס והתוכניות. לכל קורס קיימת רשימה עם פוינטר לכל התוכניות שמבצעות את הקורס ולהפך. קשר זה מאפשר לקורס בודד להשתייך למספר תוכניות לימוד במקביל עם מאפיינים שונים לכל תוכנית וגם לכל תוכנית לימוד תהיה קשר ישיר אל הקורסים שמוכלים בה.  
ExamPeriod: מכיל את גבולות הזמן של תקופת המבחנים וממפה את התאריכים החוקיים בתוכה.  
ExamSchedule: מייצג את לוח הזמנים של המבחנים באמצעות מבנה נתונים של מפתח מורכב המשלב את תקופת המבחן והקורס יחד, המיועד למנוע דריסת נתונים כאשר אותו קורס מופיע בתקופות מבחנים שונות.

## 4. שכבת האלגוריתם והאופטימיזציה:

MatchCoursesToPeriod: מחלק את בעיית השיבוץ לתתי בעיות לפי סמסטרים ומועדים.  
ConstraintIndex: בונה אינדקס מראש של כל קבוצות קורסי החובה לפי מפתח ייחודי המורכב מקוד התוכנית, שנת הלימוד והסמסטר כדי לאפשר בדיקת התנגשויות מהירה.  
BacktrackingSolver: אלגוריתם בקטראקינג הבוחן רקורסיבית את כל אפשרויות השיבוץ בתוך תקופה ספציפית.  
ForwardChecker: רכיב אופטימיזציה הצופה קדימה. לאחר כל שיבוץ זמני, הוא בודק האם לקורסים שנותרו יש עדיין תאריכים פנויים חוקיים. אם מתגלה מבוי סתום, האלגוריתם עוצר את הענף ישר וחוסך זמן ריצה.

CourseOrderingHeuristic: מיישם את כלל המשתנה המוגבל ביותר. הוא מתעדף שיבוץ של קורסים שיש להם הכי הרבה פוטנציאל להתנגשויות, מה שמוביל לחשיפה מוקדמת של אילוצים וקוטע יעיל יותר של החיפוש.

ScheduleCombiner: משלב את התוצאות המבודדות של כל תקופה באמצעות זוגות ליצירת לוחות זמנים מלאים וממוינים כרונולוגית.

### חלק III

## תהליך החשיבה, קבלת ההחלטות והקשיים בפיתוח

האלגוריתם:

האלגוריתם המרכזי של המערכת תוכנן כדי לפתור בעיה קומבינטורית קשה של התאמת קורסים לימי מבחן פנויים תחת מערכת אילוצים נוקשה. תהליך החשיבה התבסס על מעבר מגישה נאיבית ומבזבזת משאבים לגישה של Backtracking.

תהליך החשיבה והתפתחות העיצוב:

בשלב התכנון הראשוני, הגישה האינטואיטיבית הייתה לנסות ולפתור את כל מערך הקורסים והסמסטרים בבת אחת על גבי לוח שנה גלובלי יחיד. לאחר ניתוח מתמטי הבנו כי מרחב החיפוש גדל בצורה מעריכית ככל שמתווספים קורסים וימים פנויים. גישה זו הייתה גורמת לפיצוץ קומבינטורי ולזמני ריצה בלתי סבירים.

כדי להתגבר על כך, העיצוב הוחלף באסטרטגיה המורכבת משלושה שלבים מרכזיים:

- חלוקת מרחב הבעיה מראש: הבנה של האילוצים האקדמיים הראתה כי אין תלות ישירה בין תקופות מבחנים שונות (למשל, מבחן של סמסטר א' מועד א' לעולם לא יתנגש עם מבחן של סמסטר ב' מועד ב'). לכן, הצעד הראשון של האלגוריתם הוא פירוק הבעיה הגדולה למשימות שיבוץ קטנות ועצמאיות לחלוטין, המשויות לכל תקופת מבחן בנפרד.
- פתרון ממוקד ומותאם לכל תקופה: עבור כל תקופת מבחנים מבודדת, המערכת מפעילה מנגנון חיפוש עצמאי לחלוטין, מה שמקטין בצורה דרסטית את עומק וענפי עץ החיפוש.
- מיזוג והרכבה סופית: רק לאחר שנוצרו כל תתי הלוחות התקפים עבור כל התקופות בנפרד, המערכת משלבת אותם יחד ליצירת רשימה של תאריכים וקורסים המשובצים באותם התאריכים בעזרת זוגות.

המבנה הדינמי של מנוע השיבוץ:

בתוך כל תקופת מבחנים, מנוע השיבוץ פועל על בסיס שלושה רכיבים משולבים שמטרתם להגיע לפתרון חוקי:

- סידור קורסים חכם: במקום לגשת לקורסים באופן אקראי, האלגוריתם מנתח את קובץ האילוצים ומזהה אילו קורסים חולקים קבוצות חובה משותפות עם הכי הרבה

קורסים אחרים. קורסים מוגבלים אלו מקבלים עדיפות עליונה ומשובצים ראשונים. החשיבה מאחורי צעד זה היא שאם קורס קשה לשיבוץ עומד לייצר התנגשות, עדיף לגלות זאת מיד בתחילת הריצה ולא בסופה.

- חיפוש רקורסיבי עם Backtracking: האלגוריתם מתחיל לבצע שיבוצים זמניים של קורסים בימים פנויים. אם קורס מסוים משובץ ביום מסוים ולאחר מכן מתברר שלא ניתן להשלים את שאר הקורסים בצורה חוקית, המערכת מבצעת Backtrack ומוחקת את השיבוץ האחרון, חוזרת צעד אחד אחורה ומנסה תאריך חלופי.
- מנגנון צפייה קדימה ForwardChecker: כדי למנוע מהאלגוריתם לבזבז זמן על ענפי חיפוש עקרים, שולב רכיב אופטימיזציה הצופה קדימה. לאחר כל שיבוץ זמני של קורס, הרכיב סורק את כל הקורסים שנותרו לבדיקה ומוודא שלכל אחד ואחד מהם נותר לפחות תאריך פנוי חוקי אחד. אם הוא מזהה שקורס עתידי כלשהו נותר ללא תאריכים אפשריים, הוא קובע שמדובר במבוי סתום ומפסיק את החיפוש בענף זה באותו הרגע, מה שמקצץ אלפי פעולות מיותרות.

אתגרים בפרויקט:

1. כמות נתונים גדלה בקצב עצום  
הבעיה: מספר האפשרויות התיאורטי לשיבוץ קורסים גדל בצורה מעריכית על פי שילוב של כמות הימים הפנויים וכמות הקורסים. ניסיון לפתור את כל הסמסטרים והמועדים בבת אחת בעזרת BruteForce היה גורם לצריכת זיכרון עצומה וקריסת התוכנה.  
הפתרון: עיצוב רב שכבתי: פירוק הבעיה הגדולה לתתי בעיות עצמאיות לפי סמסטר ומועד, שילוב רכיב ForwardChecker שמפסיק מיידית חיפוש במסלולים שאינם מובילים לפתרון חוקי, והגדרת מגבלה קשיחה של עד 5 תוכניות לימוד בקובץ הקלט לפי דרישות התוכנה.
2. כפילות נתונים וקשרים מורכבים  
הבעיה: קורס בודד יכול להיחשב כקורס חובה עבור מסלול אחד ובמקביל כקורס בחירה עבור מסלול אחר. ייצוג שטוח ופשוט של קורסים היה מייצר כפילויות רבות בקוד ודריסת נתונים.  
הפתרון: הפרדת הישויות בקוד: קלאס Course מחזיקה רשימה פנימית של אובייקטים המייצגים את דרישות התוכנית. בנוסף, קלאס ExamPeriod משתמש במפתח מורכב המשלב את התקופה והקורס יחד כדי למנוע התנגשויות מפתחות ודריסת מידע.
3. ניהול סביבת העבודה ובדיקות אוטומטיות  
הבעיה: הבטחה ששינויים בקוד או תלות בחבילות חיצוניות לא ישברו את הלוגיקה של המערכת, ושהתוכנה תרוץ בצורה זהה אצל כל מפתח ובסביבת הייצור.  
הפתרון: בניית בדיקות Unit Tests מקיפות ושימוש ב-Docker להרצה במחשבים נפרדים בתוך Containers המאפשר להריץ את המערכת ואת כל בדיקות התוכנה בתוך סביבה מבודדת.
4. אופטימיזציה:  
הבעיה: במהלך הריצה הבנו שחלק מהמימושים שלנו לא היו מיושרים עם התוכנית

המקורית ונדרש לבצע שינויים.  
הפתרון: ביצוע Code Refactoring. לקראת אמצע הריצה בוצע שינוי ושיפור של הקבצים להבנה טובה יותר של הקוד ושלבי הריצה של התוכנית בפעולה.

5. תמיכה בשינויים בעתיד להמשך פיתוח  
הבעיה: הקוד שלנו ימשיך לגדול ולהתפתח בהמשך ונרצה קוד שהוא ניתן לשדרוג וסגור לשינויים.  
הפתרון: עבודה לפי עקרונות SOLID ומימוש של כל חלק בתוכנית בנפרד.

6. סינון ועיצוב של הגבלת השיבוץ  
הבעיה: במהלך העבודה הבנו שבהמשך נרצה לבצע שינויים וסינונים לדוגמה: שיהיה מרווח של 3 ימים בין מבחנים.  
הפתרון: שימוש בDecorator עבור אובייקט ההשוואה IsValid כד שבהמשך נוכל לבצע שינויים וסינונים שונים ולייעל את האלגוריתם בתהליכי הסינון השונים.